ACE Model (Full Data) with Mx

```
library(tinytable)
library(HDInterval)
options(tinytable_html_mathjax = TRUE)
```

```
# Define function for HDI
hdi2 <- function(x, cred_mass = 0.90) {
    hdi_interval <- HDInterval::hdi(x, credMass = cred_mass)
    setNames(hdi_interval, c("hdi_lo", "hdi_hi"))
}
```

In this note, we compare the results of the ACE model with the `umx` package in R, using the same full data as in the other note with Stan.

ACE Model

The classic model assumes that, for a given phenotype, measured by a variable Y^*, it can be decomposed into the additive genetic component A, the common environment component C, and the unique environment component E. For monozygotic (MZ) twins, both A and C are shared. Therefore, for a pair of MZ twins with outcomes Y_1^* and Y_2^*, one can express the model as

$$ Y^{*\textit{MZ}}_1 = A + C + E_1 \\ Y^{*\textit{MZ}}_2 = A + C + E_2 \\ $$

For dizygotic (DZ) twins, with only C completely shared and A shared at a half, the model is

$$ Y^{*\textit{DZ}}_1 = A_1 + C + E_1 \\ Y^{*\textit{DZ}}_2 = A_2 + C + E_2, \\ $$

where A_1 and A_2 have a correlation of 0.5. Note that measurement error is not directly incorporated in the model with a single phenotype, and the presence of random measurement will get absorbed in E.

The heretability h^2 is defined as the proportion of variance explained by A, whereas the commonality c^2 is defined as the proportion of variance explained by C. At the population level, given the twin correlations ρ^{MZ} and ρ^{DZ}, we have the following well-known result:

$$ h^2 = 2(\rho^\textit{MZ} - \rho^\textit{DZ}) \\ c^2 = 2\rho^\textit{DZ} - \rho^\textit{MZ} \\ $$

which is also described in Jöreskog (2020).

Here we do estimation using the `umx` package in R.

```
library(umx)
```

```
Loading required package: OpenMx
```

```
To take full advantage of multiple cores, use:
  mxOption(key='Number of Threads', value=parallel::detectCores()) #now
  Sys.setenv(OMP_NUM_THREADS=parallel::detectCores()) #before library(OpenMx)
```

```
Registered S3 method overwritten by 'quantmod':
  method            from
  as.zoo.data.frame zoo
```

```
For an overview type '?umx'
```

```

Attaching package: 'umx'
```

```
The following object is masked from 'package:stats':

    loadings
```

```
# Data from Jöreskog (2021)
n_mz <- c(325, 60, 7, 78, 150, 40, 8, 38, 88)
n_dz <- c(202, 97, 19, 96, 112, 64, 46, 56, 66)
```

```
# Convert to long format data
# Create contingency table
tbl_mz <- matrix(n_mz, nrow = 3, ncol = 3)
tbl_dz <- matrix(n_dz, nrow = 3, ncol = 3)

# Convert to long format for MZ
mz_table <- as.data.frame(as.table(tbl_mz))
names(mz_table) <- c("twin1", "twin2", "Freq")
mz_long <- mz_table[rep(seq_len(nrow(mz_table)), mz_table$Freq), ]
mz_long <- mz_long[, c("twin1", "twin2")]
mz_long$twin1 <- as.ordered(mz_long$twin1)
mz_long$twin2 <- as.ordered(mz_long$twin2)
mz_long$zygosity <- "MZ"
rownames(mz_long) <- NULL

# Convert to long format for DZ
```

```r
dz_table <- as.data.frame(as.table(tbl_dz))
names(dz_table) <- c("twin1", "twin2", "Freq")
dz_long <- dz_table[rep(seq_len(nrow(dz_table)), dz_table$Freq), ]
dz_long <- dz_long[, c("twin1", "twin2")]
dz_long$twin1 <- as.ordered(dz_long$twin1)
dz_long$twin2 <- as.ordered(dz_long$twin2)
dz_long$zygosity <- "DZ"
rownames(dz_long) <- NULL

# Combine both datasets
twin_data <- rbind(mz_long, dz_long)
```

```r
# Make the ordinal variables into umxFactors
twin_data[, c("twin1", "twin2")] <- umxFactor(twin_data[, c("twin1", "twin2")])

mz_data <- twin_data[twin_data$zygosity %in% "MZ", ]
dz_data <- twin_data[twin_data$zygosity %in% "DZ", ]

# Model and summary!
m1 <- umxACE(
    selDVs = "twin",
    dzData = dz_data,
    mzData = mz_data,
    sep = "",
    intervals = TRUE,
    tryHard = "ordinal"
)
```

```
Found 1 pair(s) of ordinal variables:'twin1' and 'twin2' (No binary)
```

```
Running ACE with 4 parameters
```

```

Beginning initial fit attempt
```

```
Running ACE with 4 parameters
```

```

 Lowest minimum so far:  5867.41909499216
```

```
Beginning fit attempt 1 of at maximum 10 extra tries
```

```
Running ACE with 4 parameters
```

```
 Lowest minimum so far:  5867.41909499155
```

```
Beginning fit attempt 2 of at maximum 10 extra tries
```

```
Running ACE with 4 parameters
```

```
Beginning fit attempt 3 of at maximum 10 extra tries
```

```
Running ACE with 4 parameters
```

```
Beginning fit attempt 4 of at maximum 10 extra tries
```

```
Running ACE with 4 parameters
```

```
Beginning fit attempt 5 of at maximum 10 extra tries
```

```
Running ACE with 4 parameters
```

```
 Lowest minimum so far:  5867.41909499154
```

```
Beginning fit attempt 6 of at maximum 10 extra tries
```

```
Running ACE with 4 parameters
```

```
Beginning fit attempt 7 of at maximum 10 extra tries
```

```
Running ACE with 4 parameters
```

```
Beginning fit attempt 8 of at maximum 10 extra tries
```

```
Running ACE with 4 parameters
```

```
 Lowest minimum so far:  5867.41909499153
```

```
Beginning fit attempt 9 of at maximum 10 extra tries
```

```
Running ACE with 4 parameters
```

```
Beginning fit attempt 10 of at maximum 10 extra tries
```

```
Running ACE with 4 parameters
```

```
Retry limit reached
```

```
Solution found
```

```
Final run, for Hessian and/or standard errors and/or confidence intervals
```

```
Running ACE with 4 parameters
```

```
 Solution found!  Final fit=5867.4191 (started at 5907.4077)  (11 attempt(s):
11 valid, 0 errors)
```

```
 Start values from best fit:
```

```
0.00683760097974307,0.845202329684291,0.321951358408058,0.436191539553068
```

```
ACE -2 × log(Likelihood) = 5867.419
```

```


Table: Standardized parameter estimates from a 1-factor Cholesky ACE model. A:
additive genetic; C: common environment; E: unique environment.

|     |    a1|    c1|    e1|
|:----|-----:|-----:|-----:|
|twin | 0.842| 0.321| 0.434|


Table: Means (from model$top$expMean)

|          | twin1| twin2|
|:---------|-----:|-----:|
|intercept | 0.007| 0.007|
```

```
Creating CI-based report!
```

```


|     |a1 |c1 |e1                   |
|:----|:--|:--|:--------------------|
|twin |   |   |0.434 [0.394, 0.478] |


Table: Means (from model$top$expMean)

|          | twin1| twin2|
|:---------|-----:|-----:|
|intercept | 0.007| 0.007|
```

```
?umxPlotACE options: std=T/F, means=T/F, digits=n, strip_zero=T/F, file=, min=, max =
```

```
umxSummary(m1)
```

```
ACE -2 × log(Likelihood) = 5867.42
```

```


Table: Standardized parameter estimates from a 1-factor Cholesky ACE model. A: additive genetic; C: common environment; E: unique environment.

|     |   a1|   c1|   e1|
|:----|----:|----:|----:|
|twin | 0.84| 0.32| 0.43|


Table: Means (from model$top$expMean)

|          | twin1| twin2|
|:---------|-----:|-----:|
|intercept | 0.007| 0.007|
```

```
Creating CI-based report!
```

```


|     |a1 |c1 |e1                |
|:----|:--|:--|:-----------------|
|twin |   |   |0.43 [0.39, 0.48] |


Table: Means (from model$top$expMean)

|          | twin1| twin2|
|:---------|-----:|-----:|
|intercept |  0.01|  0.01|
```

```
?umxPlotACE options: std=T/F, means=T/F, digits=n, strip_zero=T/F, file=, min=,
max =
```

```
umxConfint(m1, run = TRUE)
```

```
Running ACE with 4 parameters
```

```
               lbound estimate ubound lbound Code ubound Code
top.a_std[1,1]  0.741    0.842     NA           0          NA
top.c_std[1,1]     NA    0.321  0.499          NA           0
top.e_std[1,1]  0.394    0.434  0.478           0           0
```

Using OpenMx

Path specification

```
# Path objects for Multiple Groups
ovs <- c("twin1", "twin2")
lvs <- c("A1", "C1", "E1", "A2", "C2", "E2")
# Common paths
paths <- list(
    # variances of latent variables
    mxPath(from = lvs, arrows = 2, free = FALSE, values = 1),
    # means of latent variables
    mxPath(from = "one", to = lvs, arrows = 1, free = FALSE, values = 0),
    # means of observed variables (constrained to zero)
    mxPath(
        from = "one",
        to = ovs,
        arrows = 1,
        free = FALSE,
        values = 0,
        labels = "mean"
    ),
    # covariance between C1 & C2
    mxPath(from = "C1", to = "C2", arrows = 2, free = FALSE, values = 1),
    # path coefficients for twin 1
    mxPath(
        from = lvs[1:3],
        to = ovs[1],
        arrows = 1,
```

```
        free = TRUE,
        values = .5,
        label = c("a", "c", "e")
    ),
    # path coefficients for twin 2
    mxPath(
        from = lvs[4:6],
        to = ovs[2],
        arrows = 1,
        free = TRUE,
        values = .5,
        label = c("a", "c", "e")
    ),
    # thresholds
    mxThreshold(
        vars = ovs,
        nThresh = 2,
        free = TRUE,
        values = c(-1, 1),
        labels = c("t1", "t2")
    )
)
model_mz <- mxModel(
    "MZ",
    type = "RAM",
    manifestVars = ovs,
    latentVars = lvs,
    mxData(observed = mz_data, type = "raw"),
    paths,
    # covariance between A1 & A2 in MZ twins
    mxPath(from = "A1", to = "A2", arrows = 2, free = FALSE, values = 1)
)
model_dz <- mxModel(
    "DZ",
    type = "RAM",
    manifestVars = ovs,
    latentVars = lvs,
    mxData(observed = dz_data, type = "raw"),
    paths,
    # covariance between A1 & A2 in DZ twins
    mxPath(from = "A1", to = "A2", arrows = 2, free = FALSE, values = 0.5)
)
model_ace <- mxModel(
```

```
    "ACE",
    model_mz,
    model_dz,
    mxFitFunctionMultigroup(c("MZ", "DZ"))
)
ace_fit <- mxRun(model_ace)
```

```
Running ACE with 5 parameters
```

```
summary(ace_fit)
```

```
Summary of ACE

free parameters:
  name        matrix   row   col   Estimate Std.Error A
1    a          MZ.A twin1    A1  1.3321986 238.89659
2    c          MZ.A twin1    C1  0.5074582  91.00048
3    e          MZ.A twin1    E1  0.6875209 123.28982
4   t1 MZ.Thresholds     1 twin1 -0.1216062  21.80706
5   t2 MZ.Thresholds     2 twin1  1.3800036 247.46932

Model Statistics:
               |  Parameters  |  Degrees of Freedom  |  Fit (-2lnL units)
       Model:              5                  3099              5867.419
   Saturated:             NA                    NA                    NA
Independence:             NA                    NA                    NA
Number of observations/statistics: 1552/3104

Information Criteria:
      |  df Penalty  |  Parameters Penalty  |  Sample-Size Adjusted
AIC:      -330.5809              5877.419                5877.458
BIC:    -16901.8627              5904.156                5888.272
To get additional fit indices, see help(mxRefModels)
timestamp: 2026-10-14 01:30:46
Wall clock time: 0.05033898 secs
optimizer:  SLSQP
OpenMx version number: 2.22.11
Need help?  See help(mxSummary)
```

```
# additive genetic variance, a^2
A <- mxEval(a * a, ace_fit)
```

```
# shared environmental variance, c^2
C <- mxEval(c * c, ace_fit)
# unique environmental variance, e^2
E <- mxEval(e * e, ace_fit)
# total variance
V <- (A + C + E)
# standardized A
a2 <- A / V
# standardized C
c2 <- C / V
# standardized E
e2 <- E / V
# table of estimates
est_ace <- rbind(cbind(A, C, E), cbind(a2, c2, e2))
# likelihood of ACE model
ll_ace <- mxEval(fitfunction, ace_fit)
```

Matrix specification

```
# ACE Model
#  From  https://static1.squarespace.com/static/58b2481a9f7456906a3b9600/t/5d9c
018be77d8a0f6250b5e0/1570505099516/oneACEvo.pdf
nv <- 1 # number of phenotypes
ovs <- c("twin1", "twin2") # observed variables
nth <- 2 # number of thresholds
# Create Algebra for expected Mean & Threshold Matrices
common_list <- list(
    # Mean matrix
    mxMatrix(
        type = "Zero",
        nrow = 1,
        ncol = 2 * nv,
        name = "zero"
    ),
    # Threshold matrix
    mxMatrix(
        type = "Full",
        nrow = nth,
        ncol = 2 * nv,
        labels = rep(paste0("t", seq_len(nth)), 2),
        values = rep(c(-1, 1), 2),
        free = TRUE,
        name = "thres"
```

```
    ),
    # Create Matrices for Variance Components
    mxMatrix(
        type = "Symm",
        nrow = nv,
        ncol = nv,
        free = TRUE,
        values = 0.5,
        label = "VA11",
        name = "VA"
    ),
    mxMatrix(
        type = "Symm",
        nrow = nv,
        ncol = nv,
        free = TRUE,
        values = 0.5,
        label = "VC11",
        name = "VC"
    )
)
# Create Algebra for expected Variance/Covariance Matrices in MZ & DZ twins
covMZ <- mxAlgebra(expression = VA + VC, name = "cMZ")
covDZ <- mxAlgebra(expression = 0.5 %x% VA + VC, name = "cDZ")
expCovMZ <- mxAlgebra(
    expression = rbind(cbind(1, cMZ), cbind(t(cMZ), 1)),
    name = "expCovMZ"
)
expCovDZ <- mxAlgebra(
    expression = rbind(cbind(1, cDZ), cbind(t(cDZ), 1)),
    name = "expCovDZ"
)
# Create Data Objects for Multiple Groups
data_mz <- mxData(observed = mz_data, type = "raw")
data_dz <- mxData(observed = dz_data, type = "raw")
# Create Expectation Objects for Multiple Groups
expMZ <- mxExpectationNormal(
    covariance = "expCovMZ",
    means = "zero",
    dimnames = ovs,
    thresholds = "thres"
)
expDZ <- mxExpectationNormal(
```

```
    covariance = "expCovDZ",
    means = "zero",
    dimnames = ovs,
    thresholds = "thres"
)
funML <- mxFitFunctionML()
# Create Model Objects for Multiple Groups
model_mz <- mxModel(
    data_mz,
    common_list,
    covMZ,
    expCovMZ,
    expMZ,
    funML,
    name = "MZ"
)
model_dz <- mxModel(
    data_dz,
    common_list,
    covDZ,
    expCovDZ,
    expDZ,
    funML,
    name = "DZ"
)
multi <- mxFitFunctionMultigroup(c("MZ", "DZ"))
# Build Model with Confidence Intervals
model_ace <- mxModel(
    "ACEOrdinal",
    model_mz,
    model_dz,
    mxFitFunctionMultigroup(c("MZ", "DZ")),
    mxCI(c("MZ.VA", "MZ.VC"))
)
ace_fit <- mxRun(model_ace)
```

```
Running ACEOrdinal with 4 parameters
```

```
Warning    in    runHelper(model,    frontendStart,    intervals,    silent,
suppressWarnings, : Polite note: Model finished with a larger ordinal error than
we typically expect.
 This may be fine, but you may wish to re-run the model using
```

```
 `mxTryHardOrdinal()` in place of `mxRun()` to try for a better fit.
 Expert version: model$output[['maxRelativeOrdinalError']] is
 larger than the mvnRelEps value of  0.005 .
 If this is expected for your model, you might wish to increase `mvnRelEps`, e.g:
 mxOption(NULL, 'mvnRelEps', value= mxOption(NULL, 'mvnRelEps')*5)
 see `?mxOptions`
```

```
Warning: In model 'ACEOrdinal' Optimizer returned a non-zero status code 10.
Starting values are not feasible. Consider mxTryHard()
```

```
ace_fit <- mxTryHardOrdinal(ace_fit, intervals = TRUE)
```

```
Running ACEOrdinal with 4 parameters
```

```

Beginning initial fit attempt
```

```
Running ACEOrdinal with 4 parameters
```

```
 OpenMx status code 10 not in list of acceptable status codes, (0,1,5,6)
```

```

Beginning fit attempt 1 of at maximum 10 extra tries
```

```
Running ACEOrdinal with 4 parameters
```

```

 Lowest minimum so far:  5867.41909499156
```

```

Beginning fit attempt 2 of at maximum 10 extra tries
```

```
Running ACEOrdinal with 4 parameters
```

```

 Lowest minimum so far:  5867.41909499154
```

```
Beginning fit attempt 3 of at maximum 10 extra tries
```

```
Running ACEOrdinal with 4 parameters
```

```
Beginning fit attempt 4 of at maximum 10 extra tries
```

```
Running ACEOrdinal with 4 parameters
```

```
 Lowest minimum so far:  5867.41909499152
```

```
Beginning fit attempt 5 of at maximum 10 extra tries
```

```
Running ACEOrdinal with 4 parameters
```

```
Beginning fit attempt 6 of at maximum 10 extra tries
```

```
Running ACEOrdinal with 4 parameters
```

```
Beginning fit attempt 7 of at maximum 10 extra tries
```

```
Running ACEOrdinal with 4 parameters
```

```
Beginning fit attempt 8 of at maximum 10 extra tries
```

```
Running ACEOrdinal with 4 parameters
```

```
Beginning fit attempt 9 of at maximum 10 extra tries
```

```
Running ACEOrdinal with 4 parameters
```

```
Beginning fit attempt 10 of at maximum 10 extra tries
```

```
Running ACEOrdinal with 4 parameters
```

```
 Lowest minimum so far:  5867.4190949915
```

```
Retry limit reached
```

```
Solution found
```

```
Final run, for Hessian and/or standard errors and/or confidence intervals
```

```
Running ACEOrdinal with 4 parameters
```

```
 Solution found!  Final fit=5867.4191 (started at 7803.6412)  (11 attempt(s):
11 valid, 0 errors)
```

```
 Start values from best fit:
```

```
-0.0768346166981468,0.87192781124336,0.708498822189311,0.102801082981176
```

```
summary(ace_fit)
```

```
Summary of ACEOrdinal

free parameters:
  name   matrix row   col    Estimate  Std.Error A
1   t1 MZ.thres   1 twin1 -0.07683462 0.02694969
2   t2 MZ.thres   2 twin1  0.87192781 0.03062458
```

```
3 VA11    MZ.VA   1     1  0.70849882 0.08304975
4 VC11    MZ.VC   1     1  0.10280108 0.07757225

confidence intervals:
          lbound  estimate    ubound note
VA11  0.55000832 0.7084988 0.8761937
VC11 -0.05583271 0.1028011 0.2487363

Model Statistics:
               |  Parameters  |  Degrees of Freedom  |  Fit (-2lnL units)
       Model:              4                  3100              5867.419
   Saturated:             NA                    NA                    NA
Independence:             NA                    NA                    NA
Number of observations/statistics: 1552/3104

Information Criteria:
      |  df Penalty  |  Parameters Penalty  |  Sample-Size Adjusted
AIC:      -332.5809              5875.419                5875.445
BIC:    -16909.2100              5896.808                5884.101
To get additional fit indices, see help(mxRefModels)
timestamp: 2026-10-14 01:30:46
Wall clock time: 0.08033657 secs
optimizer:  SLSQP
OpenMx version number: 2.22.11
Need help?  See help(mxSummary)
```

Comparison with Stan results for the full data and the same 100-pair subsample used in the tutorial (55 MZ and 45 DZ pairs).

```
# Reproduce the tutorial subsample
set.seed(1234)
sub_n <- 100
twin_data_sub <- twin_data[sample.int(nrow(twin_data), sub_n), ]
m_sub <- umxACE(
    selDVs = "twin",
    dzData = twin_data_sub[twin_data_sub$zygosity == "DZ", ],
    mzData = twin_data_sub[twin_data_sub$zygosity == "MZ", ],
    sep = "",
    tryHard = "ordinal"
)
```

```
Found 1 pair(s) of ordinal variables:'twin1' and 'twin2' (No binary)
```

```
Running ACE with 4 parameters
```

```
Beginning initial fit attempt
```

```
Running ACE with 4 parameters
```

```
 Lowest minimum so far:  365.978829144973
```

```
Beginning fit attempt 1 of at maximum 10 extra tries
```

```
Running ACE with 4 parameters
```

```
Beginning fit attempt 2 of at maximum 10 extra tries
```

```
Running ACE with 4 parameters
```

```
 Lowest minimum so far:  365.978829144971
```

```
Beginning fit attempt 3 of at maximum 10 extra tries
```

```
Running ACE with 4 parameters
```

```
 Lowest minimum so far:  365.978829144971
```

```
Beginning fit attempt 4 of at maximum 10 extra tries
```

```
Running ACE with 4 parameters
```

```
 Lowest minimum so far:  365.97882914497
```

```
Beginning fit attempt 5 of at maximum 10 extra tries
```

```
Running ACE with 4 parameters
```

```
Beginning fit attempt 6 of at maximum 10 extra tries
```

```
Running ACE with 4 parameters
```

```
Beginning fit attempt 7 of at maximum 10 extra tries
```

```
Running ACE with 4 parameters
```

```
Beginning fit attempt 8 of at maximum 10 extra tries
```

```
Running ACE with 4 parameters
```

```
Beginning fit attempt 9 of at maximum 10 extra tries
```

```
Running ACE with 4 parameters
```

```
Beginning fit attempt 10 of at maximum 10 extra tries
```

```
Running ACE with 4 parameters
```

```
Retry limit reached
```

```
Solution found
```

```
Final run, for Hessian and/or standard errors and/or confidence intervals
```

```
Running ACE with 4 parameters
```

```
 Solution found!  Final fit=365.97883 (started at 370.92682)  (11 attempt(s):
11 valid, 0 errors)
```

```
 Start values from best fit:
```

```
-0.024758927371081,0.934118071479837,0,0.471878643174057
```

```
ACE -2 × log(Likelihood) = 365.979
```

```
Table: Standardized parameter estimates from a 1-factor Cholesky ACE model. A:
additive genetic; C: common environment; E: unique environment.

|     |    a1|c1 |    e1|
|:----|-----:|:--|-----:|
|twin | 0.893|.  | 0.451|


Table: Means (from model$top$expMean)

|          |  twin1|  twin2|
|:---------|------:|------:|
|intercept | -0.025| -0.025|
```

```
?umxPlotACE options: std=T/F, means=T/F, digits=n, strip_zero=T/F, file=, min=,
max =
```

OpenMx estimates and profile-likelihood intervals below are for standardized variance proportions, rather than unstandardized squared paths. An em dash indicates an unavailable interval endpoint.

```
stan_ace_fit <- readRDS(here::here("notebooks", "ace_fit_full.rds"))
stan_ace_s_fit <- readRDS(here::here("notebooks", "ace_s_fit_full.rds"))
stan_ace_res <- stan_ace_fit$summary(
    c("h2", "c2", "e2"),
    mean,
    median,
    posterior::quantile2,
    hdi2
)
stan_ace_s_res <- stan_ace_s_fit$summary(
    c("h2", "c2", "e2"),
    mean,
    median,
    posterior::quantile2,
    hdi2
)
stan_ace_sub_fit <- readRDS(here::here("notebooks", "ace_fit.rds"))
stan_ace_s_sub_fit <- readRDS(here::here("notebooks", "ace_s_fit.rds"))
stan_ace_sub_res <- stan_ace_sub_fit$summary(
    c("h2", "c2", "e2"),
    mean,
    median,
    posterior::quantile2,
    hdi2
)
stan_ace_s_sub_res <- stan_ace_s_sub_fit$summary(
    c("h2", "c2", "e2"),
    mean,
    median,
    posterior::quantile2,
    hdi2
)

ace_proportion_ci <- function(model) {
    # Request intervals directly on the proportions, not on path coefficients.
    if (length(model$intervals) > 0) {
        model <- mxModel(model, mxCI(names(model$intervals)), remove = TRUE)
    }
    model <- mxModel(
        model,
```

```
        mxAlgebra(top.A / top.Vtot, name = "h2_ace"),
        mxAlgebra(top.C / top.Vtot, name = "c2_ace"),
        mxAlgebra(top.E / top.Vtot, name = "e2_ace")
    )
    targets <- c("h2_ace[1,1]", "c2_ace[1,1]", "e2_ace[1,1]")
    model_ci <- umxCI(model, which = targets, run = "yes", interval = 0.90)
    targets <- paste0(model_ci$name, ".", targets)
    res <- model_ci$output$confidenceIntervals[
        targets,
        c("estimate", "lbound", "ubound")
    ]
    codes <- model_ci$output$confidenceIntervalCodes[targets, , drop = FALSE]
    for (bound in c("lbound", "ubound")) {
        failed <- is.na(codes[, bound]) |
            codes[, bound] != 0 |
            !is.finite(res[, bound])
        res[failed, bound] <- NA_real_
    }
    rownames(res) <- c("h2", "c2", "e2")
    res
}
umx_res <- ace_proportion_ci(m1)
```

```
Running ACE with 4 parameters
```

```
                lbound estimate ubound lbound Code ubound Code
ACE.h2_ace[1,1]  0.575    0.708     NA           0          NA
ACE.c2_ace[1,1]     NA    0.103  0.226          NA           0
ACE.e2_ace[1,1]  0.160    0.189  0.221           0           0
[1] "NA: "
```

```
Table: Computed CIs in model ACE
```

```
umx_sub_res <- ace_proportion_ci(m_sub)
```

```
Running ACE with 4 parameters
```

```
                lbound estimate ubound lbound Code ubound Code
ACE.h2_ace[1,1]     NA    0.797     NA          NA          NA
ACE.c2_ace[1,1]     NA    0.000  0.261          NA           3
```

```
ACE.e2_ace[1,1]     NA    0.203     NA          NA          NA
[1] "NA: "
[1] "3: The nonlinear constraints and bounds could not be satisfied. The problem
may have no feasible solution."
```

```
Table: Computed CIs in model ACE
```

```
drop_leading_zero <- function(x) sub("^(-?)0\\.", "\\1.", x)

fmt_num <- function(x) drop_leading_zero(sprintf("%.3f", x))

fmt_ci <- function(lo, hi) {
    fmt_bound <- function(x) {
        if (is.na(x)) "\u2014" else fmt_num(x)
    }
    sprintf("[%s, %s]", fmt_bound(lo), fmt_bound(hi))
}

comparison_columns <- function(mx_res, rho_res, simplex_res) {
    rows <- lapply(c("h2", "c2", "e2"), function(parameter) {
        rho <- rho_res[rho_res$variable == parameter, ]
        simplex <- simplex_res[simplex_res$variable == parameter, ]
        data.frame(
            Estimate = fmt_num(c(
                mx_res[parameter, "estimate"],
                rho$mean,
                simplex$median
            )),
            `90% CI` = c(
                fmt_ci(
                    mx_res[parameter, "lbound"],
                    mx_res[parameter, "ubound"]
                ),
                fmt_ci(rho$q5, rho$q95),
                fmt_ci(simplex$hdi_lo, simplex$hdi_hi)
            ),
            check.names = FALSE
        )
    })
    do.call(rbind, rows)
}
```

```
comp_tbl <- data.frame(
    Model = rep(c("OpenMx", "Stan (unconstrained)", "Stan (simplex)"), 3),
    comparison_columns(umx_res, stan_ace_res, stan_ace_s_res),
    comparison_columns(umx_sub_res, stan_ace_sub_res, stan_ace_s_sub_res),
    check.names = FALSE
)

tt(comp_tbl) |>
    group_tt(
        i = list("$h^2$" = 1, "$c^2$" = 4, "$e^2$" = 7),
        j = list(
            "Full data (N = 1,552)" = 2:3,
            "Subsample (N = 100)" = 4:5
        )
    )
```

	Full data (N = 1,552)		Subsample (N = 100)	
Model	Estimate	90% CI	Estimate	90% CI
h^2				
OpenMx	.708	[.575, —]	.797	[—, —]
Stan (unconstrained)	.710	[.576, .850]	1.194	[.604, 1.822]
Stan (simplex)	.698	[.587, .810]	.631	[.342, .847]
c^2				
OpenMx	.103	[—, .226]	.000	[—, —]
Stan (unconstrained)	.098	[-.034, .221]	-.426	[−1.019, .126]
Stan (simplex)	.109	[.002, .202]	.110	[.000, .327]
e^2				
OpenMx	.189	[.160, .221]	.203	[—, —]
Stan (unconstrained)	.192	[.163, .225]	.233	[.123, .380]
Stan (simplex)	.191	[.161, .221]	.241	[.106, .390]